

MonopolyLK Verification Technique

Automated Log Parsing, State-Space Auditing, and Multi-Agent Behavioral Analysis Guidelines

Target System:	MONOPOLY-LK (C-Based Multi-Agent Simulation)	Agents Audited:	Banker, Risk Taker, Aggressive, Trader
Methodology:	Programmatic Trace Parsing & Line-Level Verification	Audit Engine:	Large Language Model (Gemini / Claude Code Interpreter)

1. Executive Summary & Objective

The **MonopolyLK Verification Technique** establishes a rigorous workflow for identifying subtle logical defects, memory corruption artifacts, dynamic ledger discrepancies, and state-space inconsistencies in C-based multi-agent simulations.

In complex simulations where multiple autonomous entities (e.g., *Conservative Banker*, *Risk Taker*, *Aggressive Investor*, and *Opportunistic Trader*) interact dynamically, standard debugging or manual log inspection often fails to catch race conditions, orphaned asset states, or minor balance drifts. This protocol leverages standard Unix output redirection alongside LLM-driven programmatic analysis to systematically audit stdout logs.

2. Execution Protocol Workflow

To execute this verification technique, follow the three-step compilation, execution, and analysis workflow outlined below:

Step	Phase	Command / Tool	Description
Step 1	Compilation	<code>gcc *.c -o monopoly</code>	Compile all source C files into a single binary executable using GCC. Ensure clean compilation without syntax errors.
Step 2	Execution & Redirection	<code>./monopoly > run.txt</code>	Run the simulation executable and redirect standard output (<code>stdout</code>) into the file <code>run.txt</code> for evaluation.
Step 3	AI Analysis	Upload <code>run.txt</code> to Gemini / Claude	Submit the generated trace log to the AI model using the exact verification prompt specified in Section 3.

Execution Note: Ensure that full logging options are enabled in your C codebase prior to Step 1. The output log (`run.txt`) must capture granular state changes, roll outcomes, property purchases, trades, and elimination lifecycle events.

3. The Exact Verification Prompt

When submitting the stdout log (`run.txt`) to Gemini or Claude, use the exact structured prompt below without altering its core instruction set:

TARGET PROMPT — COPY & PASTE INTO AI INTERFACE

I've attached the stdout log from a run of a modified Monopoly simulation ("MONOPOLY-LK", written in C – gcc *.c -o monopoly). Please write and run code to analyze it for bugs and anomalies. Don't just skim it – parse it programmatically and verify claims with evidence (line numbers).

My simulation has these entities/agents:

Conservative Banker
Risk Taker
Aggressive Investor
Opportunistic Trader

Please check for the following categories of problems. Adapt each one to whatever the equivalent concept is in my simulation:

State-space consistency – do fixed reference values (board squares, node IDs, category labels, etc.) ever map to inconsistent names/meanings at different points in the log?
Anything that should be a constant but isn't.

Ownership / assignment integrity

Does any resource, item, or role ever get attributed to the wrong owner/agent relative to the last transaction that assigned it?

Does any agent interact with something in a way that shouldn't be possible given who currently owns/controls it (e.g. paying itself, acting on a resource it doesn't hold)?

Does any eliminated/inactive/removed agent still take actions after being removed?

Lifecycle / elimination handling – this is the area most likely to hide a real bug in multi-agent simulations:

Once an agent is removed, bankrupted, or marked "done," do its resources/assets/properties get properly reassigned, released, or cleaned up – or do they become orphaned?

Search every later reference to something owned by a removed agent. If it still shows up as "owned" by them with no further activity (e.g. permanently zero value, unreachable, frozen), that's a strong signal of a missing cleanup/handoff step.

Right before an agent is removed/eliminated, does the log show the actual numbers that triggered it, or does it skip straight to the elimination message with no evidence of what caused it?

Ledger / running-total spot-checks – pick several individual transactions or state changes and manually verify that the "before" and "after" values in the log are internally consistent (previous value + delta = new value). Important: if my simulation has legitimate dynamic mechanics (inflation, market events, randomness, decay, etc.), don't flag a changing value as a bug on its own – only flag it if a specific transaction's math doesn't check out.

Final/summary arithmetic – if the log has an end-of-run summary with component values (e.g. cash + assets – debts = net worth, or any other "total should equal the sum of parts" claim), verify the arithmetic explicitly and flag any mismatch with the exact numbers.

Range/bounds sanity – are randomly generated or bounded values (dice rolls, percentages, indices, etc.) ever outside their valid range?

Anything else structurally inconsistent – e.g., a status flag that should suppress an action but doesn't, an action that occurs without the expected preceding trigger, or

values changing with no logged transaction to explain it.
For each finding:
State it as a clear, one-sentence claim.
Back it with exact line number(s) from the log and a short quoted excerpt as evidence.
Clearly separate confirmed bugs (reproducible, evidence-backed) from things that looked odd but turned out to be legitimate simulation behavior – don't report the latter as bugs.

4. Detailed Category Explanations & Verification Mechanics

To assist software engineers and researchers in interpreting results, the target evaluation categories are expanded below:

1. State-Space Consistency

Validates that static simulation parameters (e.g., Board Square IDs 0–39, Color Group names, Tax values) remain constant across time steps. Catches pointer corruption or memory overwrites where constant strings or IDs drift during execution.

2. Ownership & Assignment Integrity

Tracks state transitions of properties and assets across agents. Verifies that agents cannot pay rent to themselves, trade unowned assets, or execute decisions after bankruptcy flags are raised.

3. Lifecycle & Elimination Handling

Audits agent death/bankruptcy cleanup pipelines. Checks whether assets owned by eliminated agents (e.g., *Risk Taker*) are properly auctioned, returned to the bank, or granted to creditors, preventing "ghost owner" syndromes in subsequent turns.

4. Ledger & Running-Total Spot-Checks

Programmatically parses delta transitions: $\text{ext}\{\text{Balance}\}_{t+1} = \text{ext}\{\text{Balance}\}_t + \Delta$. Distinguishes true arithmetic bugs from intentional simulation dynamics like dynamic tax rates or inflation events.

5. Final / Summary Arithmetic

Verifies the terminal state report at simulation end. Ensures that reported Net Worth ($\text{ext}\{\text{Net Worth}\} = \text{ext}\{\text{Cash}\} + \text{ext}\{\text{Asset Value}\} - \text{ext}\{\text{Liabilities}\}$) mathematically balances across all surviving and eliminated agent records.

6. Range & Bounds Sanity

Audits array index bounds, dice roll ranges ($\$2 \leq \text{ext}\{\text{Roll}\} \leq 12$), percentage caps ($\$0\% - 100\%$), and board position wrapping logic ($\$0 \leq \text{ext}\{\text{Pos}\} \leq 39$).

5. Expected AI Finding Output Format

When the AI analyzes `run.txt` using script execution (e.g., Python code execution), findings are structured strictly as follows to prevent false positives:

[CONFIRMED BUG]

Claim: The Risk Taker agent was charged rent on Square 14 after being eliminated on Line 412.

Evidence: Line 523 -> "Agent Risk Taker pays \$50 rent to Conservative Banker for Square 14."

[VERIFIED LEGITIMATE BEHAVIOR]

Observation: Agent Cash increased by \$200 on Square 0 without a transaction log entry.

Explanation: Verified as standard rule mechanics (Passing G0 salary collected automatically).